

Swinburne University of Technology
School of Science, Computing, and Emerging Technologies

SWE40006 - Software Deployment and Evolution
Deployment Portfolio Task 3

Student name: Ngoc Van Nhi Do
Student ID: 105551765
Submission date: DON'T FORGET THISSSSSSSSSSSS

Deployment Portfolio Task 3

1 Declared Target Level

All four subtasks 3.1, 3.2, 3.3, and 3.4 were attempted.

Table 1 below contain relevant links as proof of successful task execution.

Table 1: Publicly accessible URLs for live grading verification

Source	URL
ALB DNS Name	http://task-3-3-alb-2099008439.ap-southeast-2.elb.amazonaws.com/

2 Execution Evidence

2.1 Task 3.1

For this task, I deployed a WordPress application onto an Amazon EC2 instance.

2.1.1 AWS Environment Setup

First, I created a new AWS account on the free plan, which gave me \$100 in free credits.

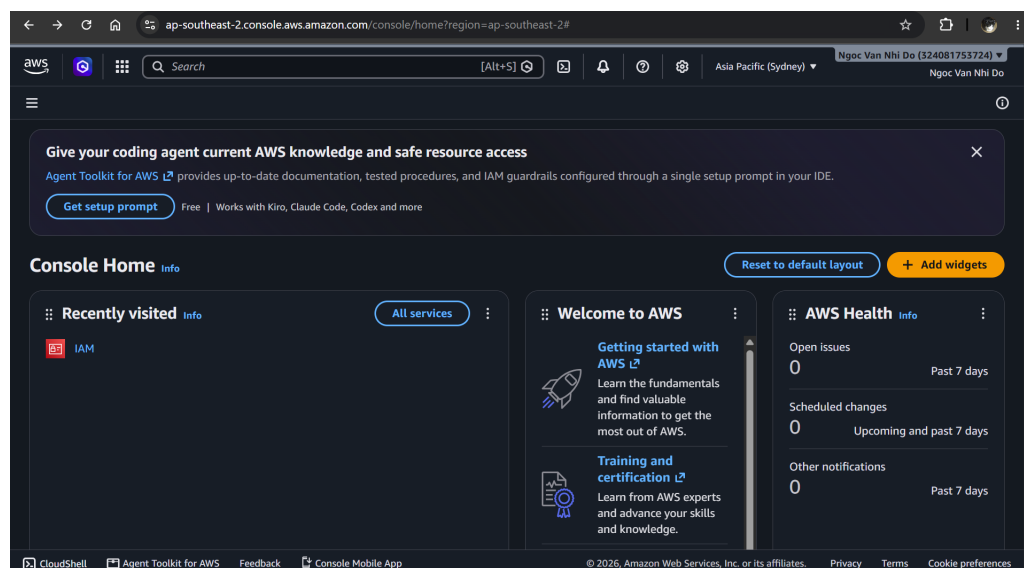


Figure 1: The AWS Console interface after I had successfully signed up.

I then created a new IAM user for my coursework named `SWE40006_student` rather than using the root account. This user is assigned to a new user group `EC2-Lab-Users` which is given full access to EC2 resources (using `AmazonEC2FullAccess`).

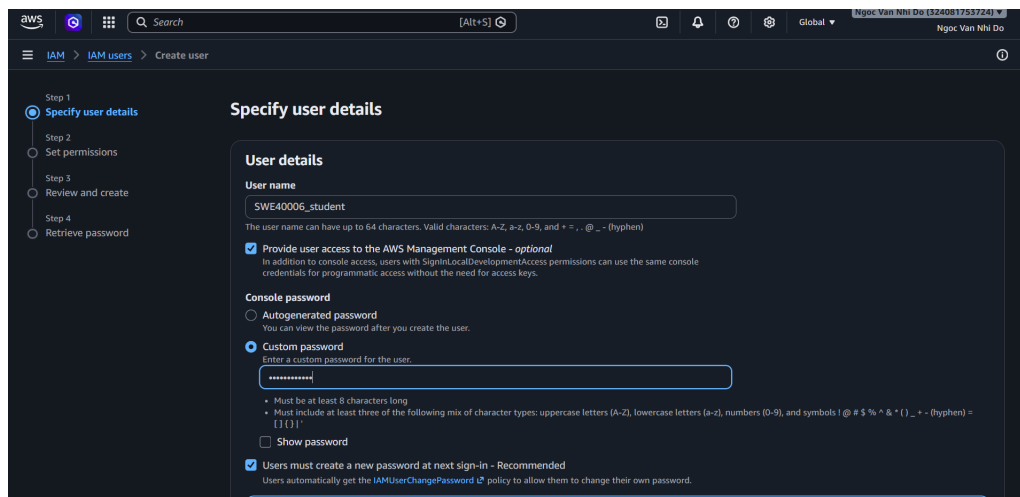


Figure 2: Creating a new IAM user SWE40006_student.

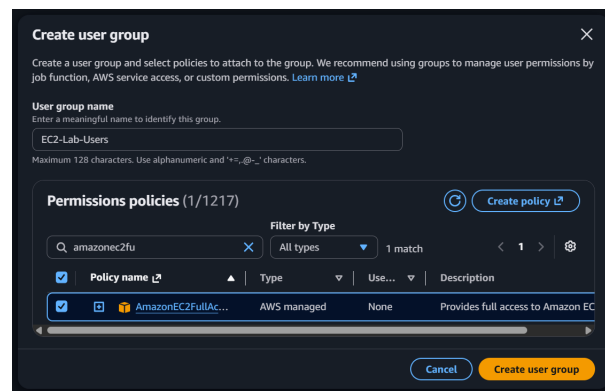


Figure 3: Adding AmazonEC2FullAccess to the user group EC2-Lab-Users, which is then assigned to SWE40006_student.

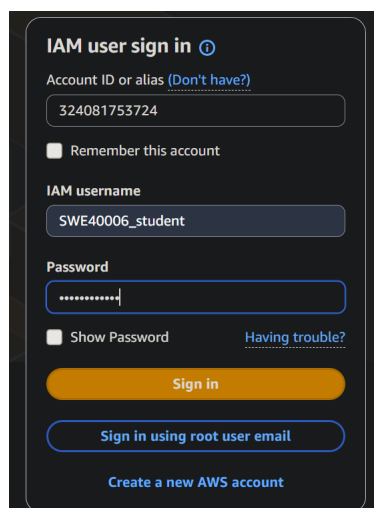
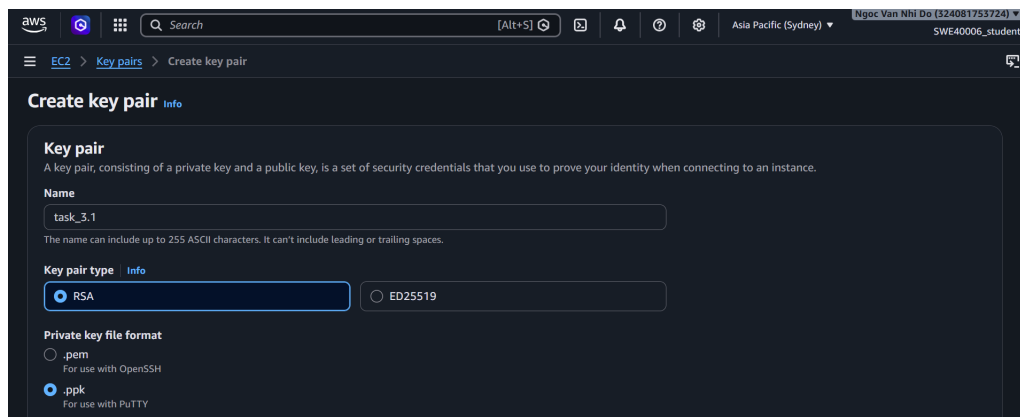


Figure 4: Logging into the AWS Console as the new IAM user.

To securely login into my EC2 instances, I must generate an SSH key pair. The private key was downloaded and stored on my local machine for later authentication.

Figure 5: Creating a new key pair `task-3.1`.

2.1.2 Compute Provisioning

I launched a new EC2 instance called `Task 3.1`. This instance runs Amazon Linux 2023, uses a `t3.micro` instance type to remain eligible for the free tier, and is configured with the previously generated key pair.

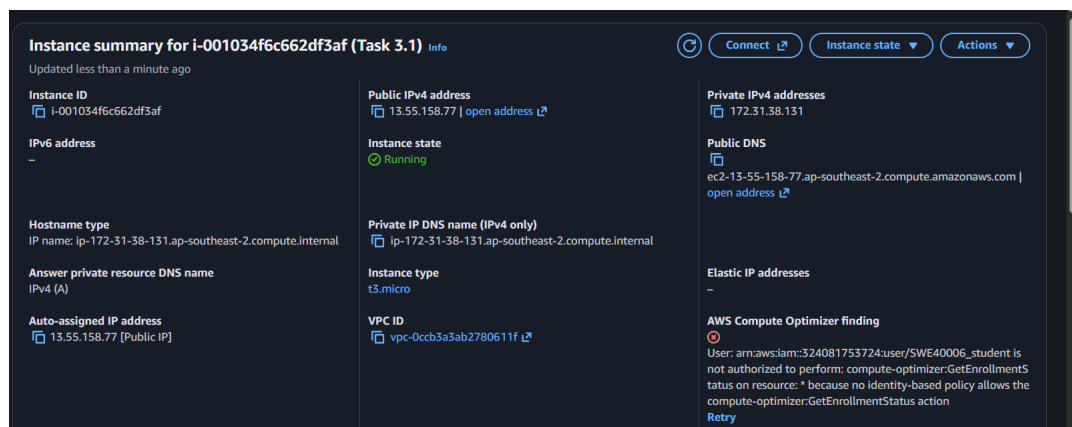


Figure 6: Creating a new Amazon EC2 instance.

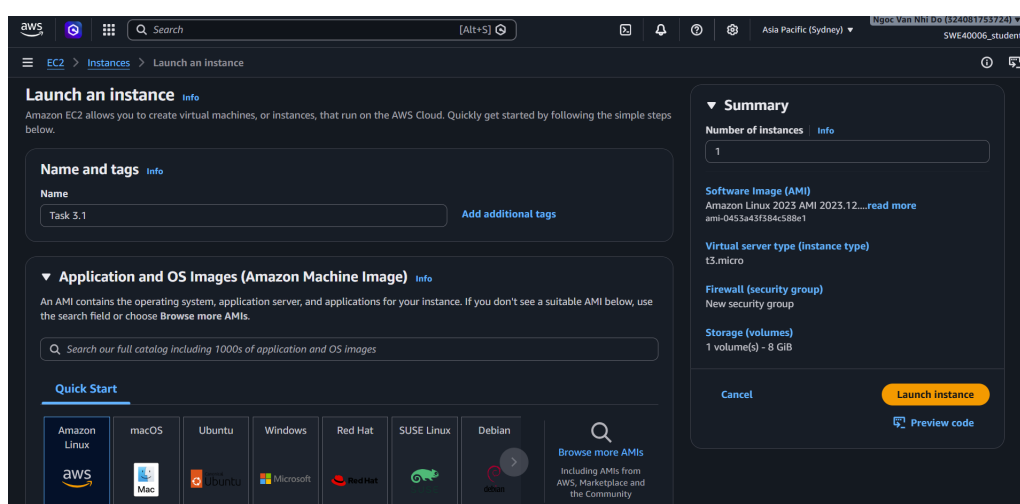
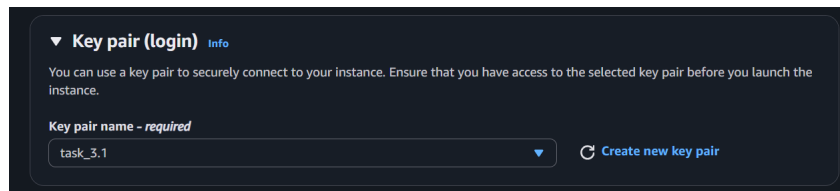


Figure 7: Amazon Linux 2023 was chosen as the OS for this instance.

Figure 8: Configuring the instance with the `task-3.1` key pair.

I then configured the instance with a new security group `task-3.1-sg`, which allowed SSH (22), HTTP (80), and HTTPS (443) inbound traffic from anywhere.

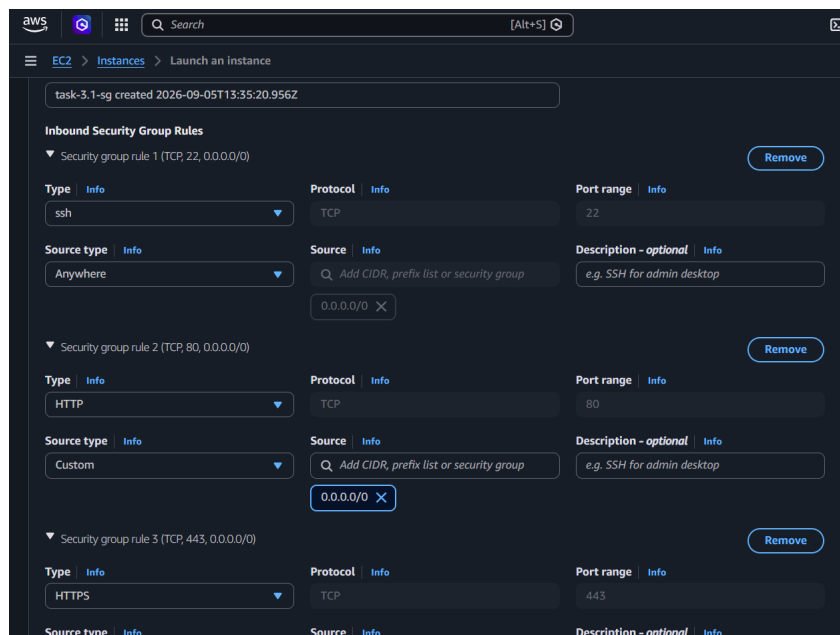


Figure 9: Configuring the security rules to allow inbound SSH, HTTP, and HTTPS traffic to this instance.

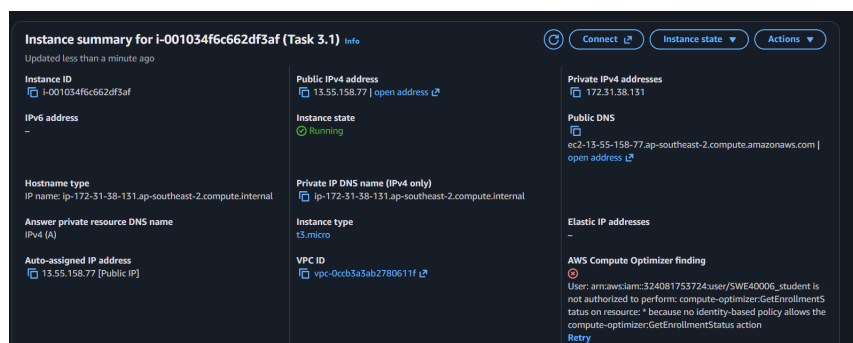


Figure 10: The new instance is successfully created.

I then SSH-ed to the instance using PuTTY.

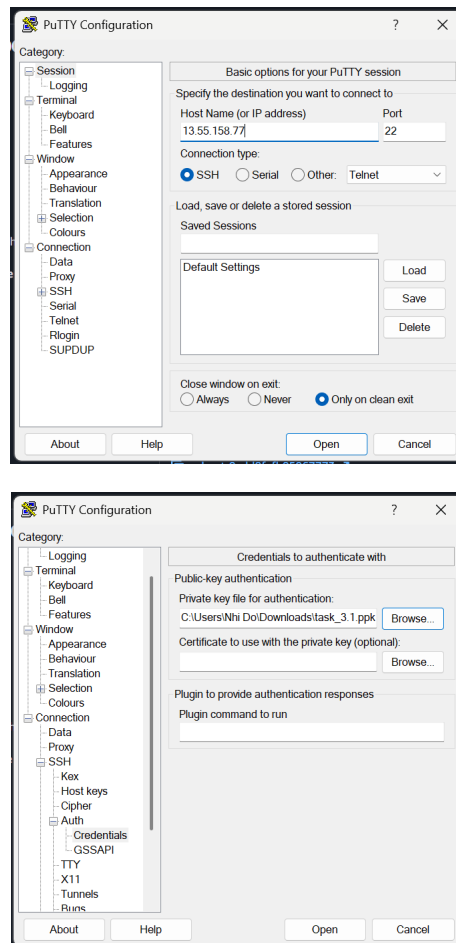


Figure 11: Connecting to the EC2 instance using SSH.



Figure 12: Connection was successful.

I then installed the Apache web server and started it so that it could later host my web application.

```

ec2-user@ip-172-31-38-131:~$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
ec2-user@ip-172-31-38-131:~$ sudo dnf install httpd -y
Last metadata expiration check: 0:00:08 ago on Sat Sep 5 15:19:40 2026.
Dependencies resolved.

=====
Package                Arch      Version                Repository              Size
=====
Installing:
httpd                  x86_64    2.4.68-1.amzn2023.0.1  amazonlinux             46 k
Installing dependencies:
apr                    x86_64    1.7.5-1.amzn2023.0.4   amazonlinux             129 k
apr-util               x86_64    1.6.5-1.amzn2023.0.1   amazonlinux             100 k
apr-util-ldap          x86_64    1.6.5-1.amzn2023.0.1   amazonlinux             15 k
generic-logos-httpd    noarch    18.0.0-12.amzn2023.0.3  amazonlinux             19 k
httpd-core             x86_64    2.4.68-1.amzn2023.0.1  amazonlinux             1.4 M
httpd-filesystem       noarch    2.4.68-1.amzn2023.0.1  amazonlinux             12 k
httpd-tools            x86_64    2.4.68-1.amzn2023.0.1  amazonlinux             80 k
libbrotli              x86_64    1.0.9-4.amzn2023.0.2   amazonlinux             315 k
mailcap                noarch    2.1.49-3.amzn2023.0.3  amazonlinux             33 k
Installing weak dependencies:
apr-util-openssl       x86_64    1.6.5-1.amzn2023.0.1   amazonlinux             17 k
mod_http2              x86_64    2.0.42-1.amzn2023.0.1  amazonlinux             167 k
mod_lua                x86_64    2.4.68-1.amzn2023.0.1  amazonlinux             59 k
=====
Transaction Summary
=====
Install 13 Packages

Total download size: 2.4 M
Installed size: 7.0 M
Downloading Packages:
(1/13): apr-util-1.6.5-1.amzn2023.0.1.x86_64.rpm      2.7 MB/s | 100 kB  00:00
(2/13): apr-util-ldap-1.6.5-1.amzn2023.0.1.x86_64.rpm 376 kB/s | 15 kB  00:00
(3/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm          2.8 MB/s | 129 kB  00:00
(4/13): apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64.rpm 731 kB/s | 17 kB  00:00
(5/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch. 805 kB/s | 19 kB  00:00
(6/13): httpd-2.4.68-1.amzn2023.0.1.x86_64.rpm      1.6 MB/s | 46 kB  00:00
Complete!
ec2-user@ip-172-31-38-131:~$ sudo systemctl start httpd
ec2-user@ip-172-31-38-131:~$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
ec2-user@ip-172-31-38-131:~$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-09-05 15:20:34 UTC; 13s ago
     Docs: man:httpd.service(8)
    Main PID: 28911 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0"
    Tasks: 177 (limit: 1059)
   Memory: 13.7M
      CPU: 69ms
   CGroup: /system.slice/httpd.service
           └─28911 /usr/sbin/httpd -DFOREGROUND
             └─28912 /usr/sbin/httpd -DFOREGROUND
               └─28913 /usr/sbin/httpd -DFOREGROUND
                 └─28915 /usr/sbin/httpd -DFOREGROUND
                   └─28920 /usr/sbin/httpd -DFOREGROUND

Sep 05 15:20:34 ip-172-31-38-131.ap-southeast-2.compute.internal systemd[1]: Starting httpd.service: The Apache HTTP Server.
Sep 05 15:20:34 ip-172-31-38-131.ap-southeast-2.compute.internal systemd[1]: Started httpd.service: The Apache HTTP Server.
Sep 05 15:20:34 ip-172-31-38-131.ap-southeast-2.compute.internal httpd[28911]: Server configured.
ec2-user@ip-172-31-38-131:~$

```

Figure 13: Installing and starting the web server.

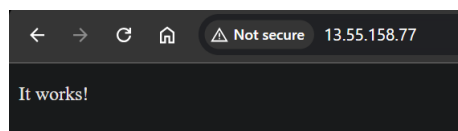


Figure 14: Verifying that the web server is publicly accessible.

2.1.3 Application Deployment

Before deploying the WordPress application, I needed to install its dependencies which included installing PHP and MariaDB.

```

ec2-user@ip-172-31-38-131:~$ sudo dnf install php php-mysqlnd php-gd php-mbstring php-xml php-json php-curl php-zip -y
Last metadata expiration check: 1:23:32 ago on Sat Sep 5 15:19:40 2026.
Dependencies resolved.

```

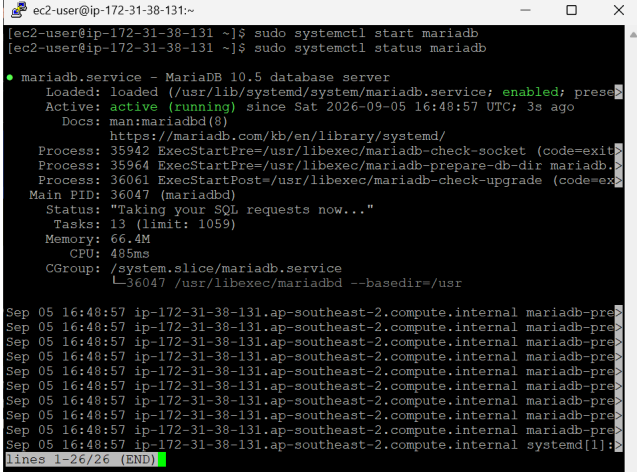
Figure 15: Installing PHP.

```

ec2-user@ip-172-31-38-131:~$ sudo dnf install -y mariadb105-server
Last metadata expiration check: 1:27:01 ago on Sat Sep 5 15:19:40 2026.
Dependencies resolved.

```

Figure 16: Installing MariaDB.



```

ec2-user@ip-172-31-38-131:~
[ec2-user@ip-172-31-38-131 ~]$ sudo systemctl start mariadb
[ec2-user@ip-172-31-38-131 ~]$ sudo systemctl status mariadb

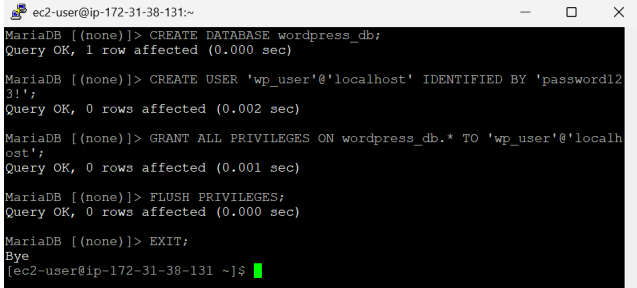
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; prese
   Active: active (running) since Sat 2026-09-05 16:48:57 UTC; 3s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 35942 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exit
   Process: 35964 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.>
   Process: 36061 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=ex
   Main PID: 36047 (mariadb)
    Status: "Taking your SQL requests now..."
     Tasks: 13 (limit: 1059)
    Memory: 66.4M
       CPU: 485ms
   CGroup: /system.slice/mariadb.service
           └─36047 /usr/libexec/mariadb --basedir=/usr

Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal systemd[1]:
lines 1-26/26 (END)

```

Figure 17: Starting and enabling the MariaDB service.

I then set up a local database for the web application.



```

MariaDB [(none)]> CREATE DATABASE wordpress_db;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'password123!';
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost';
Query OK, 0 rows affected (0.001 sec)

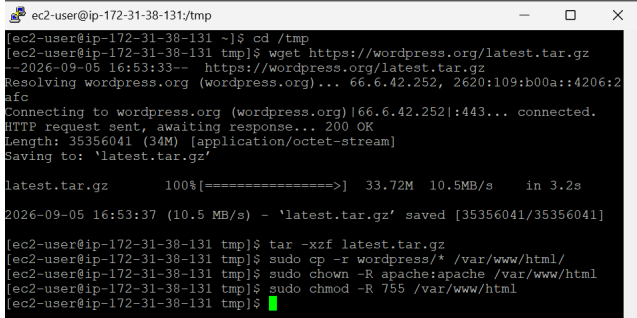
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> EXIT;
Bye
[ec2-user@ip-172-31-38-131 ~]$

```

Figure 18: Setting up a local database named `wordpress_db` for the WordPress application.

I installed and extracted the WordPress package. The command `chown` made Apache the owner of the WordPress files, while `chmod` set who could read, write, or access those files. These commands ensured that Apache had the correct permissions to run WordPress.



```

ec2-user@ip-172-31-38-131/tmp
[ec2-user@ip-172-31-38-131 ~]$ cd /tmp
[ec2-user@ip-172-31-38-131 tmp]$ wget https://wordpress.org/latest.tar.gz
--2026-09-05 16:53:33-- https://wordpress.org/latest.tar.gz
Resolving wordpress.org (wordpress.org)... 66.6.42.252, 2620:109:b00a::4206:2
afe
Connecting to wordpress.org (wordpress.org)|66.6.42.252|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 35356041 (34M) [application/octet-stream]
Saving to: 'latest.tar.gz'

latest.tar.gz      100%[=====>]  33.72M  10.5MB/s  in 3.2s
2026-09-05 16:53:37 (10.5 MB/s) - 'latest.tar.gz' saved [35356041/35356041]

[ec2-user@ip-172-31-38-131 tmp]$ tar -xzf latest.tar.gz
[ec2-user@ip-172-31-38-131 tmp]$ sudo cp -r wordpress/* /var/www/html/
[ec2-user@ip-172-31-38-131 tmp]$ sudo chown -R apache:apache /var/www/html
[ec2-user@ip-172-31-38-131 tmp]$ sudo chmod -R 755 /var/www/html
[ec2-user@ip-172-31-38-131 tmp]$

```

Figure 19: Installing WordPress and configuring permissions.

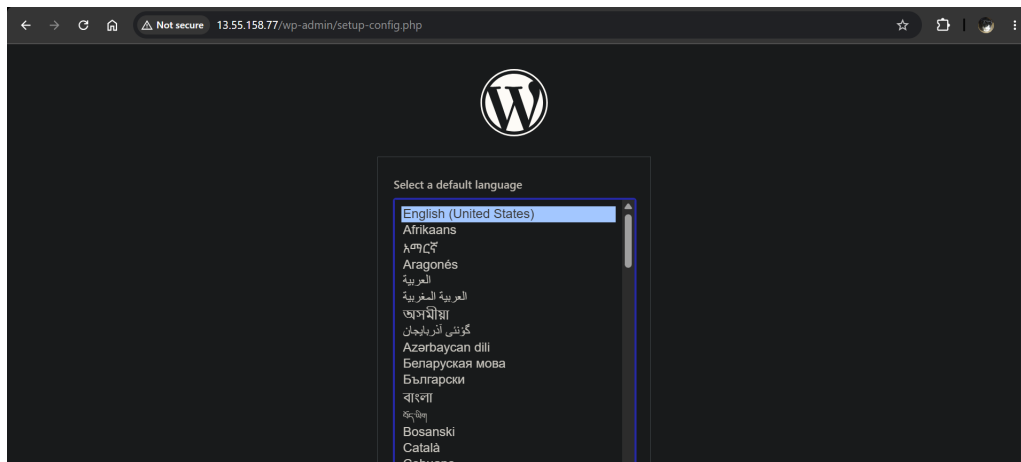


Figure 20: The deployed WordPress application.

2.2 Task 3.2

For this task, I configured my web application to use a database hosted on an Amazon RDS instance, effectively decoupling the database from the EC2 instance. I also created a backup of my web application by storing my files on an S3 bucket, and demonstrated how I could restore my application using this backup.

2.2.1 Database Decoupling

I started by creating a new RDS instance and configured it to use MariaDB as the database engine. I also connected the EC2 instance to this database right away, so that AWS automatically allowed inbound traffic on port 3306 from the web application to the database for me.

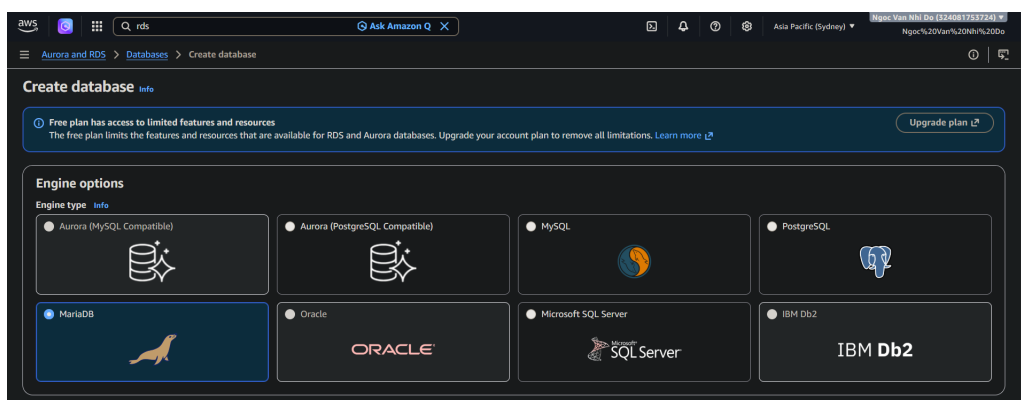


Figure 21: Creating a new RDS instance.

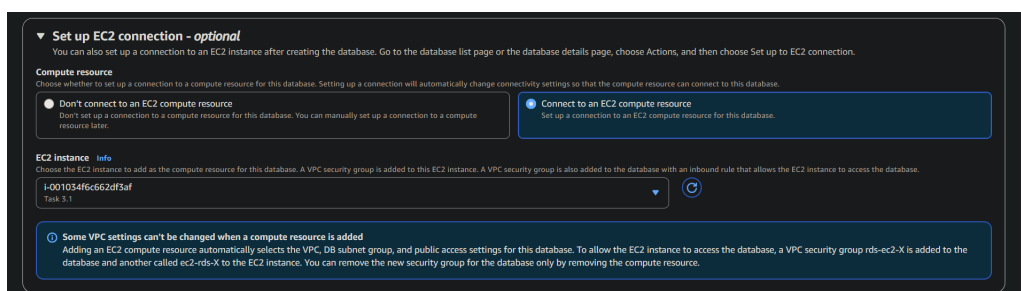


Figure 22: Connecting the EC2 instance from the previous task.

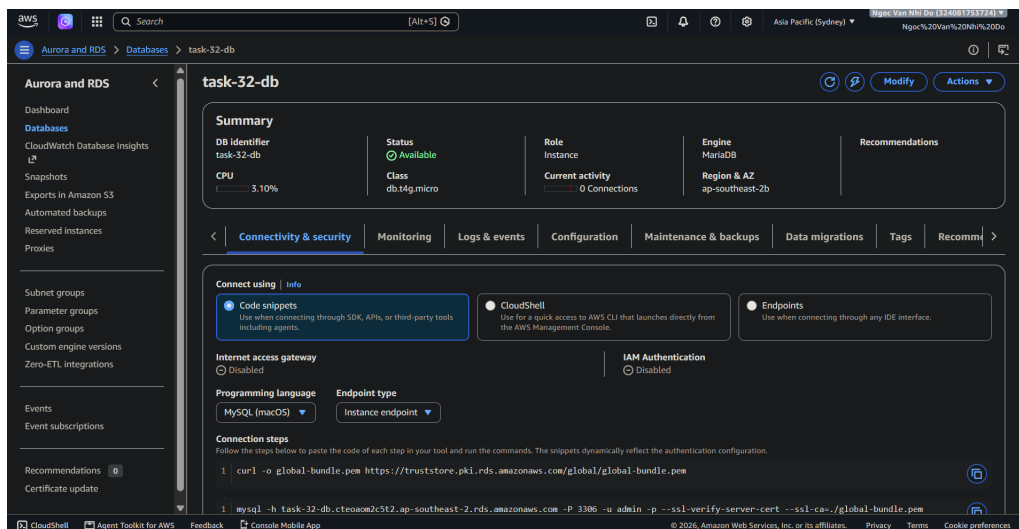


Figure 23: The database is successfully created.

From the EC2 instance's command line, I established a connection to the RDS instance using its endpoint at `task-32-db.cteoam2c5t2.ap-southeast-2.rds.amazonaws.com`.

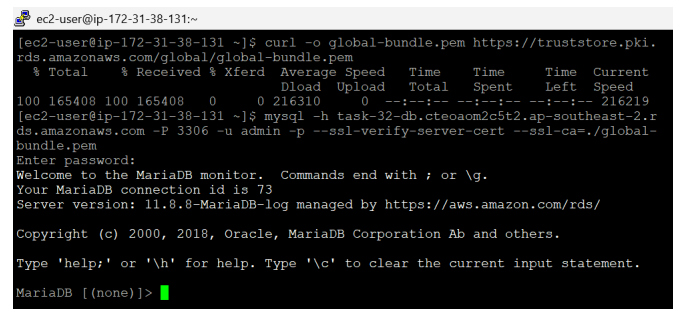


Figure 24: Connecting to the database from the EC2 instance's terminal.

Once I could connect to the database, I created a new database `wordpress_db` to be used by my application. I then exited the DB connection, configured the `[wg-config.php]` file with the required database information, including the database name, username, and password, along with the other necessary credentials. This allowed the WordPress application to connect to and use the configured database. Finally, I restarted the Apache web server and verified that the application has been successfully configured to use the database. As I had not installed the actual WordPress application by this point, the database displayed all empty entries.

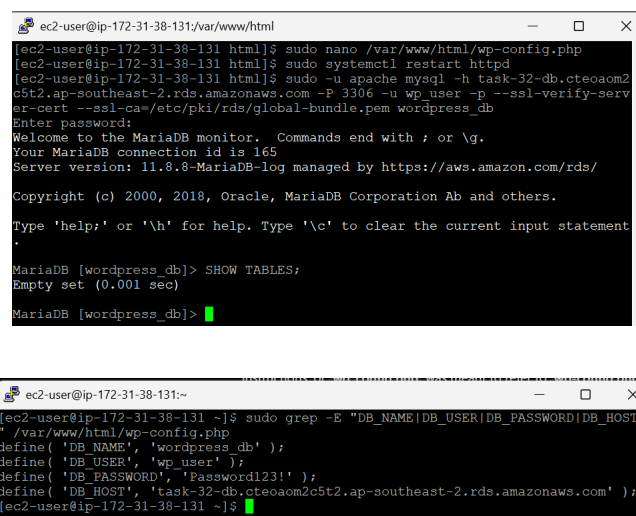


Figure 25: Editing the database configurations and verifying the connection to the RDS database.

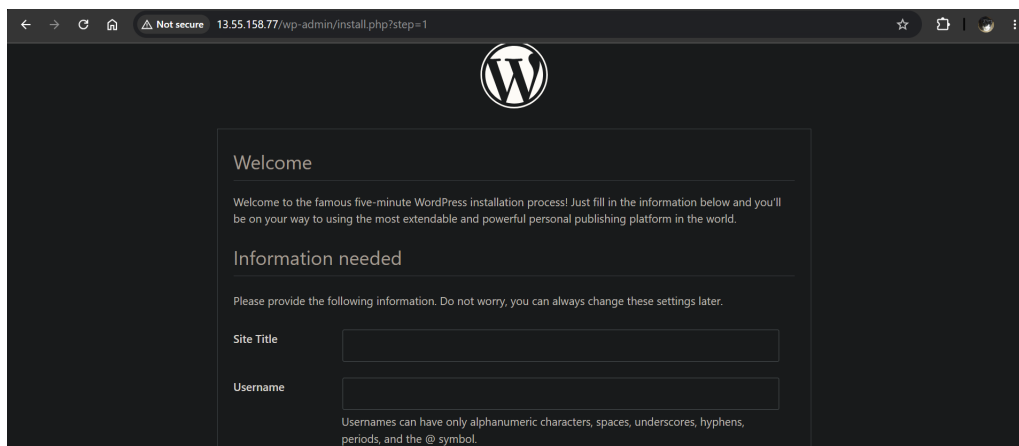


Figure 26: The application now shows the WordPress installation form, which means the database has successfully been connected.

2.2.2 Backup and Restore

I started by creating a new S3 bucket with the name `swe40006-105551765-bucket` to store my backup files.

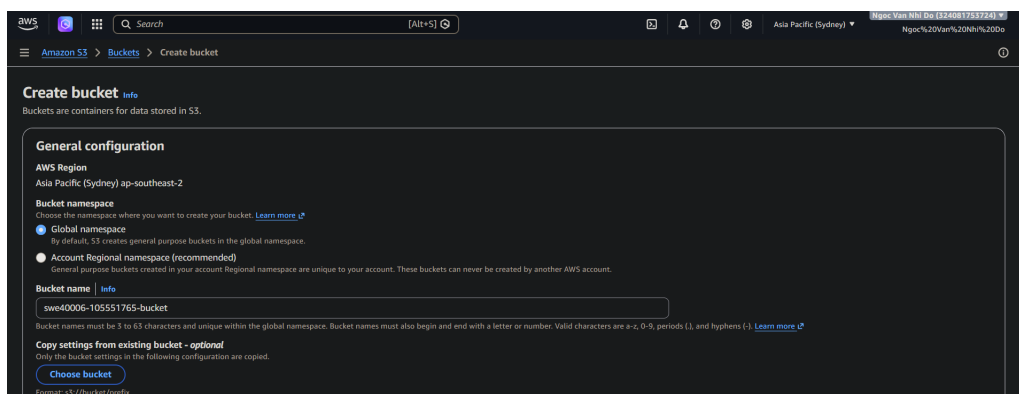


Figure 27: The application now shows the WordPress installation form, which means the database has successfully been connected.

I also created a new IAM role allowing full access to S3. This role is then attached to my EC2 instance so that the server could read and write files to the S3 bucket.

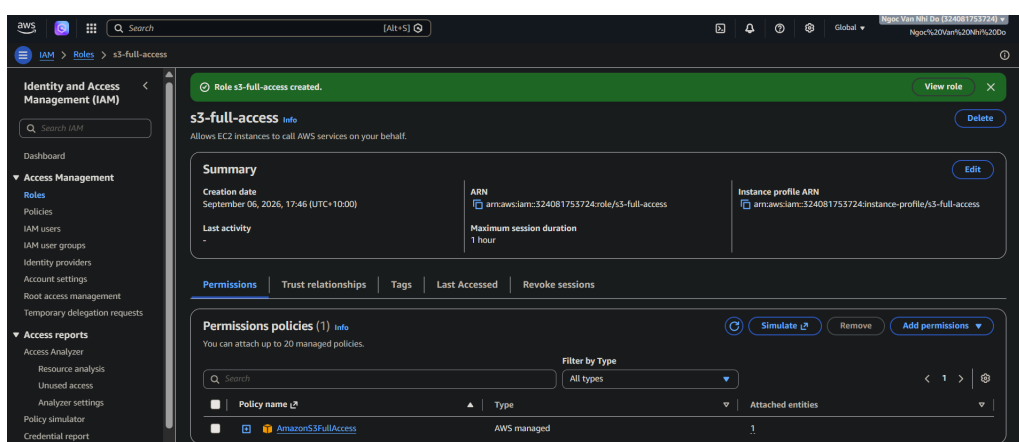


Figure 28: Creating a new IAM role with the AmazonS3FullAccess policy.



Figure 29: Adding the role to my EC2 instance.

I then made a backup of my files by archiving them into a `.tar` file to avoid having to individually handle hundreds of different files. This archive is then uploaded to the S3 bucket, serving as the backup to my web server.

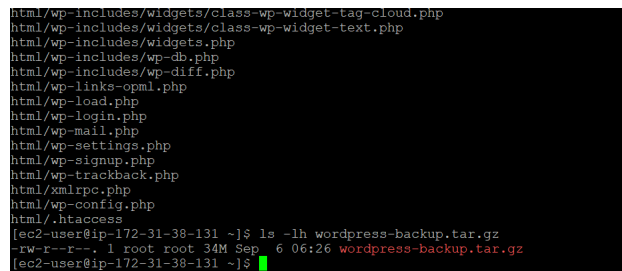
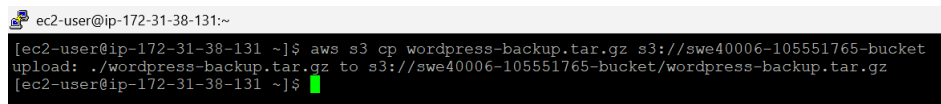
Figure 30: Creating the backup as a `.tar` archive.

Figure 31: Uploading the backup to the S3 bucket.

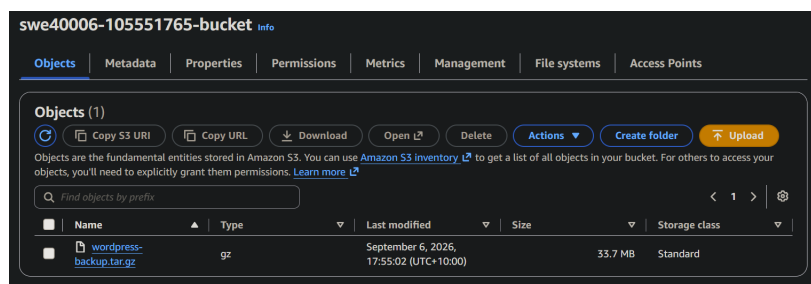


Figure 32: I could see that the backup has been uploaded using the AWS Console.

To demonstrate that I can restore my web server, I first removed all my web application files to simulate a deletion.

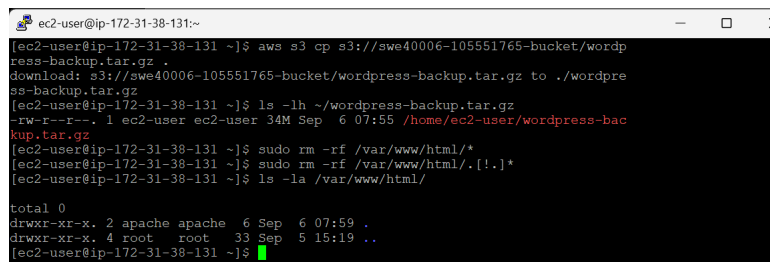


Figure 33: Removing all web application files on my EC2 instance.

Once the files were removed, accessing my website displayed only an "It works!" message rather than the WordPress application, which is the default Apache web server page. This confirmed that the web application had been completely deleted.

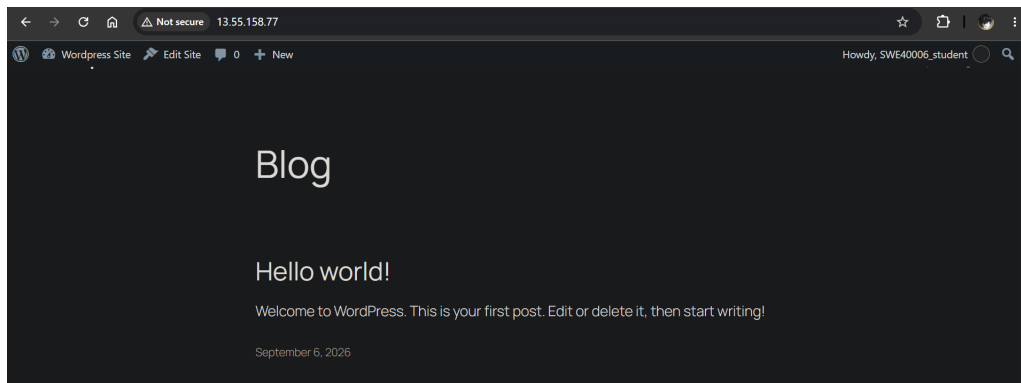


Figure 34: The WordPress application had been deleted.

To restore my deleted web application, I pulled the backup from the S3 bucket and extracted the archived files. The final step was to modify Apache's permissions to be able to read these files and restarting the web server.

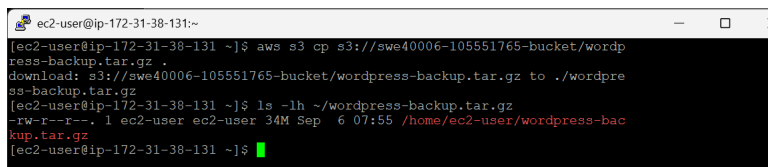


Figure 35: Downloading the backup from the S3 bucket.

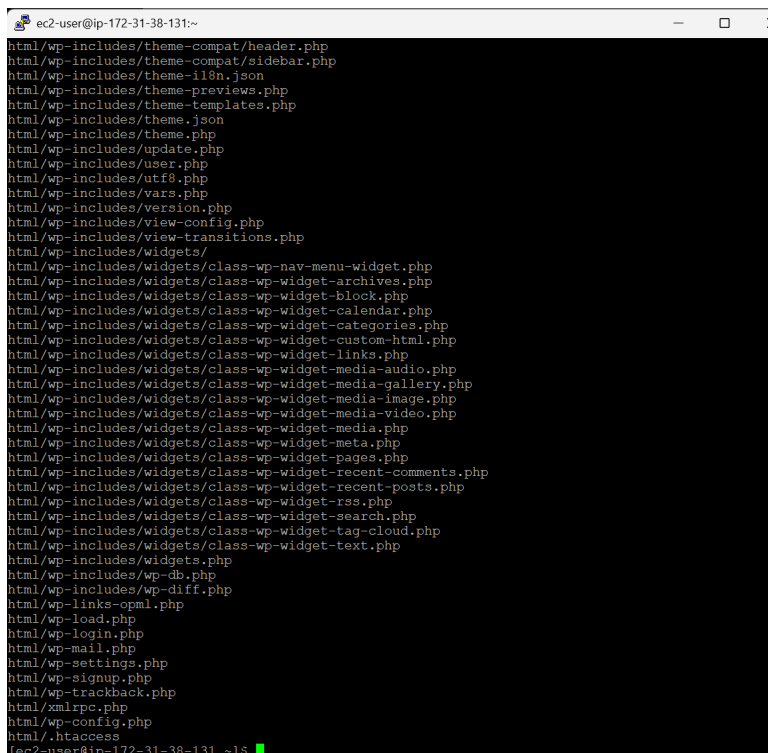
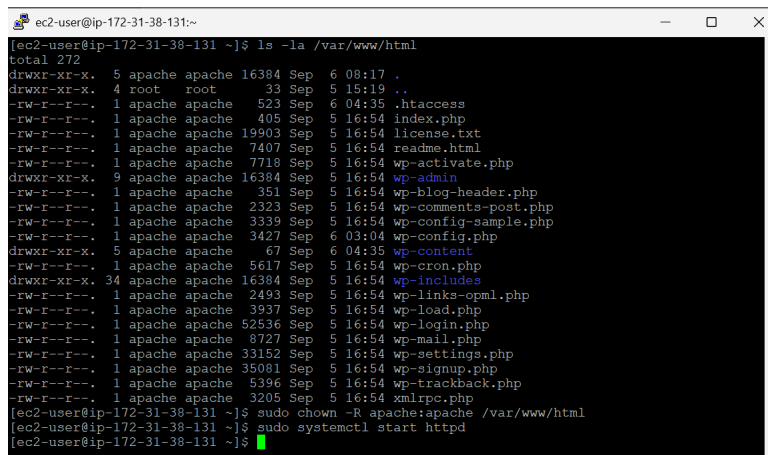


Figure 36: Extracting the web application files.



```

ec2-user@ip-172-31-38-131:~$ ls -la /var/www/html
total 272
drwxr-xr-x. 5 apache apache 16384 Sep  6 08:17 .
drwxr-xr-x. 4 root root 33 Sep  5 15:19 ..
-rw-r--r--. 1 apache apache 523 Sep  6 04:35 .htaccess
-rw-r--r--. 1 apache apache 405 Sep  5 16:54 index.php
-rw-r--r--. 1 apache apache 19903 Sep  5 16:54 license.txt
-rw-r--r--. 1 apache apache 7407 Sep  5 16:54 readme.html
-rw-r--r--. 1 apache apache 7718 Sep  5 16:54 wp-activate.php
drwxr-xr-x. 9 apache apache 16384 Sep  5 16:54 wp-admin
-rw-r--r--. 1 apache apache 351 Sep  5 16:54 wp-blog-header.php
-rw-r--r--. 1 apache apache 2323 Sep  5 16:54 wp-comments-post.php
-rw-r--r--. 1 apache apache 3339 Sep  5 16:54 wp-config-sample.php
-rw-r--r--. 1 apache apache 3427 Sep  6 03:04 wp-config.php
drwxr-xr-x. 5 apache apache 67 Sep  6 04:35 wp-content
-rw-r--r--. 1 apache apache 5617 Sep  5 16:54 wp-cron.php
drwxr-xr-x. 34 apache apache 16384 Sep  5 16:54 wp-includes
-rw-r--r--. 1 apache apache 2493 Sep  5 16:54 wp-links-opml.php
-rw-r--r--. 1 apache apache 3937 Sep  5 16:54 wp-load.php
-rw-r--r--. 1 apache apache 52536 Sep  5 16:54 wp-login.php
-rw-r--r--. 1 apache apache 8727 Sep  5 16:54 wp-mail.php
-rw-r--r--. 1 apache apache 33152 Sep  5 16:54 wp-settings.php
-rw-r--r--. 1 apache apache 35081 Sep  5 16:54 wp-signup.php
-rw-r--r--. 1 apache apache 5396 Sep  5 16:54 wp-trackback.php
-rw-r--r--. 1 apache apache 3205 Sep  5 16:54 xmlrpc.php
[ec2-user@ip-172-31-38-131 ~]$ sudo chown -R apache:apache /var/www/html
[ec2-user@ip-172-31-38-131 ~]$ sudo systemctl start httpd
[ec2-user@ip-172-31-38-131 ~]$

```

Figure 37: Verifying that the application files have been extracted.

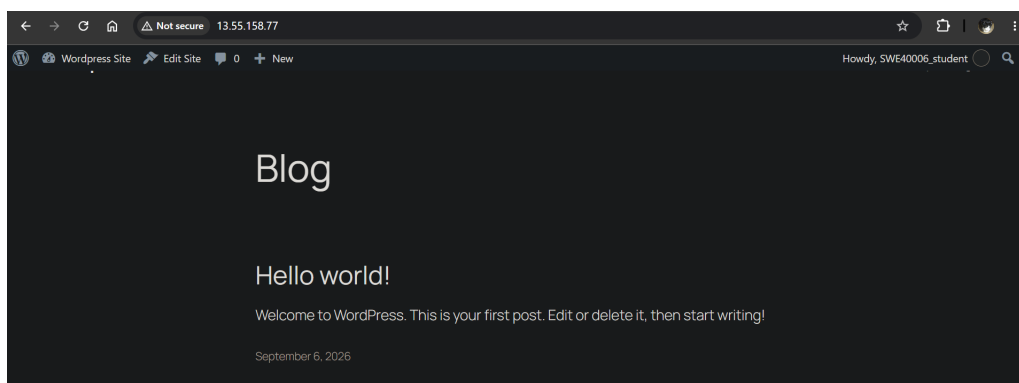


Figure 38: The web application was successfully restored.

2.3 Task 3.3

For this task, I created an EC2 Launch Template containing all the necessary configurations to recreate the web server and application on my original EC2 instance. I also provisioned an Application Load Balancer (ALB) across the **ap-southeast-2a** and **ap-southeast-2b** availability zones to route public HTTP/HTTPS traffic. The load balancer will route traffic to a target group managed by an Auto Scaling Group (ASG). The ASG uses the Launch Template to configure EC2 instances and maintains the desired number of instances, launching new instances in the event of instance failures.

2.3.1 Creating the Launch Template

A new launch template (**task-3-3-template**) is created and configured with the Amazon Linux AMI, the **t3.micro** instance type, and the security group **task-3.1-sg** that was used for the original EC2 instance.

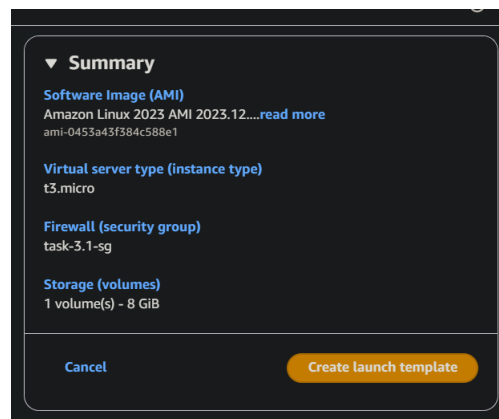


Figure 39: Summary of the launch template's configurations.

Configuring the user data for this template is especially important, as it is the startup script that runs when the instance launches, ensuring that everything necessary is correctly installed and configured to serve our web application. The script below updates the OS, installs the web server, PHP, and AWS CLI, starts Apache and PHP_FPM, creates the WordPress web directory, downloads the backup that I created earlier from S3, extracts the files, sets file permissions, and restarts the web server.

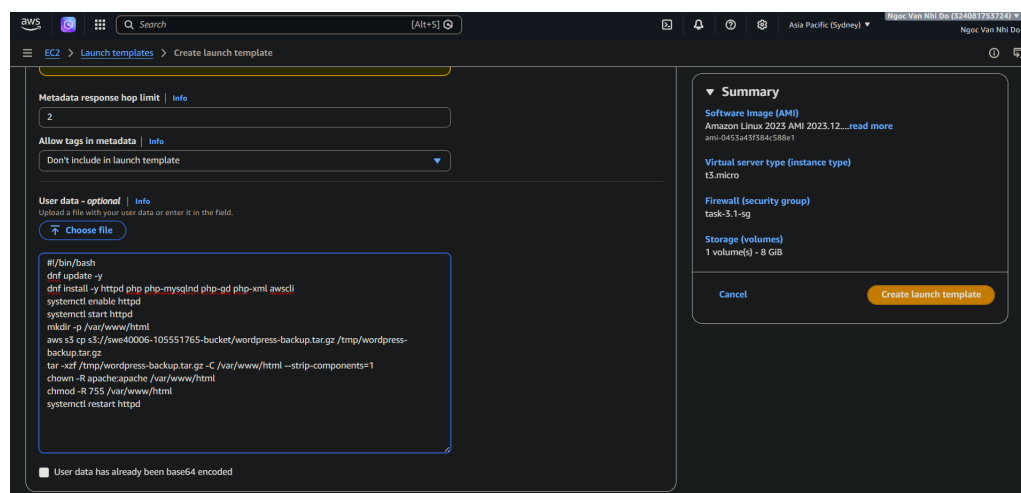


Figure 40: The template's user data script.

2.3.2 Creating the Application Load Balancer (ALB)

Before creating the ALB, I had to set up a new security group for it. The idea is to put the ALB in front of my application instances and make the instances private from the public internet. The ALB will handle incoming HTTP/HTTPS traffic from users on the internet, then distribute these requests across the application instances.

The ALB's security group will allow inbound HTTP/HTTPS traffic from anywhere.

Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)
task-3.3-sg
Name cannot be edited after creation.

Description [Info](#)
Allows HTTP and HTTPS access from anywhere

VPC [Info](#)
vpc-0ccb3a3ab2780611f

Inbound rules [Info](#)

Type	Protocol	Port range	Source	Description - optional	
HTTP	TCP	80	Any...	0.0.0.0/0	Delete
HTTPS	TCP	443	Any...	0.0.0.0/0	Delete

[Add rule](#)

Figure 41: Creating a new security group for the ALB.

The inbound rules for the security group that was used for the original EC2 instance were also changed to only receive HTTP traffic from the ALB's security group. This also implies that the web application hosted on the first EC2 instance is no longer publicly accessible.

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules [Info](#)

Security group rule ID	Type	Protocol	Port range	Source	Description - optional	
sgr-0388515bb12fa35c3	SSH	TCP	22	Custom	0.0.0.0/0	Delete
-	HTTP	TCP	80	Custom	sg-02fdd0a1ab200ad1a	Delete

[Add rule](#) [Cancel](#) [Preview changes](#) [Save rules](#)

Figure 42: Configuring the rules for the application instances' security group to no longer allow HTTP traffic from the internet.

I then created an ALB (**task-3-3-alb**) spanning the availability zones **ap-southeast-2a** and **ap-southeast-2b**, configured with the security group **task-3.3-sg** that was just created above.

Create Application Load Balancer [Info](#)

The Application Load Balancer distributes incoming HTTP and HTTPS traffic across multiple targets such as Amazon EC2 instances, microservices, and containers, based on request attributes. When the load balancer receives a connection request, it evaluates the listener rules in priority order to determine which rule to apply, and if applicable, it selects a target from the target group for the rule action.

How Application Load Balancers work

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.
task-3-3-alb
A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme [Info](#)
Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**

- Serves internet-facing traffic.
- Has public IP addresses.
- DNS name resolves to public IPs.
- Requires a public subnet.

☐ **Internal**

- Serves internal traffic.
- Has private IP addresses.
- DNS name resolves to private IPs.
- Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type [Info](#)
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ **IPv4**
Includes only IPv4 addresses.

Figure 43: Creating a new ALB with the name **task-3-3-alb**.

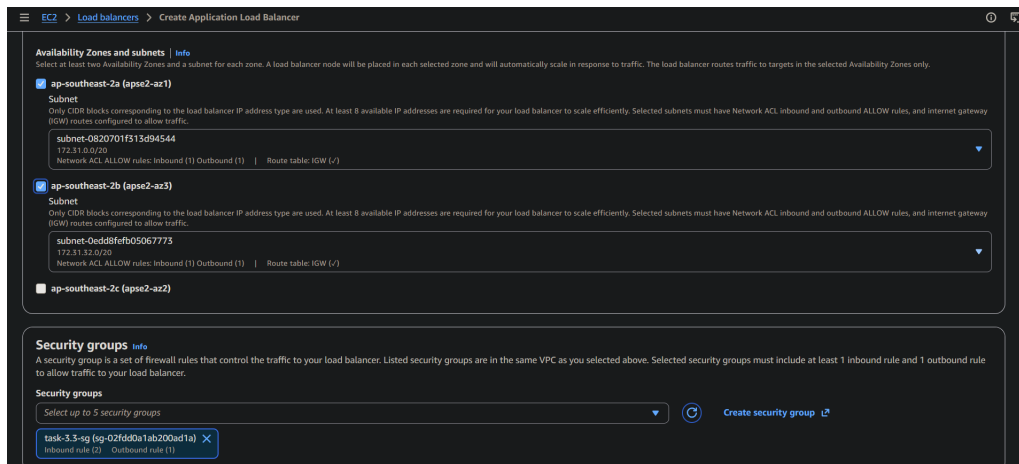
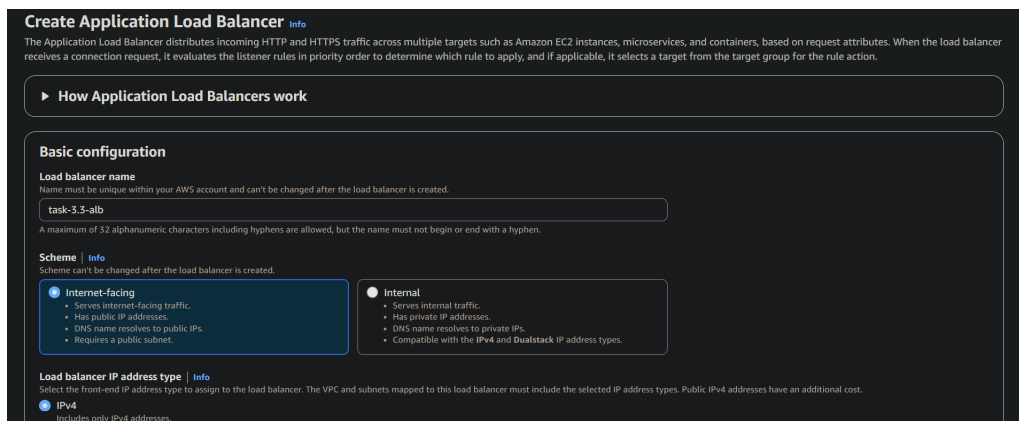
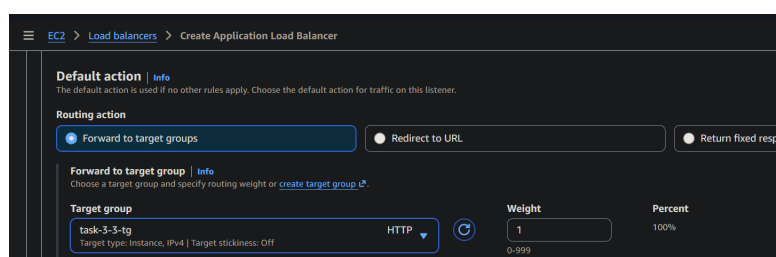


Figure 44: Configuring the ALB's availability zones and security group.

I also created a new target group that would receive traffic from the ALB. A health check was configured on port 80 to verify that the instances would be running and responding correctly. The target group was then attached to the ALB, allowing the ALB to forward incoming traffic to healthy instances.

Figure 45: Creating a new target group `task-3-3-tg`. The target group is not registered with any instances for now.Figure 46: Attaching `task-3-3-tg` to the ALB.

2.3.3 Creating the Auto Scaling Group (ASG)

I then created a new ASG with the following configurations:

- Name: `task-3-3-asg`
- Launch template: `task-3-3-template`
- Availability Zones: `ap-southeast-2a` and `ap-southeast-2b` (matching the ALB's availability zones)
- Load balancer: `task-3-3-alb`
- Target group: `task-3-3-tg`
- Desired capacity: 2
- Minimum capacity: 2

- Maximum capacity: 4

The figure consists of three screenshots from the AWS Management Console, showing the configuration steps for an Auto Scaling Group (ASG).

Step 1: Choose launch template

Group details

Auto Scaling group name: task-3-3-asg

Launch template

Launch template	Version	Description
task-3-3-template	Default	

Step 2: Choose instance launch options

Network

VPC: vpc-0cc3a3ab2780611f

Availability Zones and subnets

Availability Zone	Subnet	Subnet CIDR range
ap-se2-az1 (ap-southeast-2a)	subnet-0820701f313d94544	172.31.0.0/20
ap-se2-az3 (ap-southeast-2b)	subnet-0edd88febf05067773	172.31.32.0/20

Step 3: Integrate with other services

Load balancing

Load balancer 1

Name	Type	Target group
task-3-3-alb	Application/HTTP	task-3-3-tg

Step 4: Configure group size and scaling policies

Group size

Desired capacity	Desired capacity type
2	Units (number of instances)

Scaling

Minimum desired capacity	Maximum desired capacity
2	4

Target tracking policy: -

Figure 47: A summary of ASG's configurations.

2.3.4 Verification

Once everything had been set up and configured, I noticed that the health checks for my instances were returning an "Unhealthy" status. I initially suspected that the issue was related to networking for traffic being unable to reach the instances correctly. I began investigating the VPC configuration, security groups, inbound and outbound security rules, and the availability zones to ensure that everything was correctly configured. After confirming that these settings were correct, I decided to SSH into one of the failing instances to further troubleshoot this issue.

The screenshot shows the 'Registered targets' tab for a target group. It displays a table with three instances, each with a health status of 'Unhealthy'.

Instance ID	Name	Port	Zone	Health status	Health status details	Admin...
i-02fac9721d555c062	-	80	ap-southeast-...	Unhealthy	Request timed out	No ov...
i-08972a025c345aac8	-	80	ap-southeast-...	Draining	Target deregistration i...	No ov...
i-049ce36eb4707c081	-	80	ap-southeast-...	Unhealthy	Request timed out	No ov...

Figure 48: Health checks were returning an "Unhealthy" status, causing the ASG to deregister failing instances and launch new ones to replace them.

I investigated a wide range of potential issues, checking installed dependencies and making sure they were running, verifying that the web application files were properly installed into the correct directory, or that the database configurations contained the correct information. Testing showed that Apache and PHP-FPM were running correctly and that PHP scripts could execute successfully. The WordPress installation was also correctly restored from the S3 backup, including the `wp-config.php` file, which pointed to the RDS database. The root cause was an incorrect RDS security group configuration that prevented the EC2 instances from connecting to the database on port 3306. When the ALB requests /, WordPress tried to connect to the database and hung waiting for the database connection, which made Apache eventually time out.

```
ec2-user@ip-172-31-44-220:~$ php -v
PHP 8.5.9 (cli) (built: Jul 28 2026 13:06:52) (NTS gcc x86_64)
Copyright (c) The PHP Group
Built by Amazon Linux
Zend Engine v4.5.9, Copyright (c) Zend Technologies
with Zend OPcache v8.5.9, Copyright (c), by Zend Technologies
ec2-user@ip-172-31-44-220:~$ sudo systemctl status httpd --no-pager
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
           └─php-fpm.conf
   Active: active (running) since Mon 2026-09-07 03:25:54 UTC; 20min ago
     Docs: man:httpd.service(8)
   Main PID: 11062 (httpd)
   Status: "Total requests: 94; Idle/Busy workers 100/0;Requests/sec: 0.0764; Bytes served/sec: 1.9KB/sec"
     Tasks: 230 (limit: 1059)
    Memory: 21.1M
       CPU: 1.55s
   CGroup: /system.slice/httpd.service
           └─11062 /usr/sbin/httpd -DFOREGROUND
             11163 /usr/sbin/httpd -DFOREGROUND
             11170 /usr/sbin/httpd -DFOREGROUND
             11185 /usr/sbin/httpd -DFOREGROUND
             11191 /usr/sbin/httpd -DFOREGROUND
             26360 /usr/sbin/httpd -DFOREGROUND

Sep 07 03:25:54 ip-172-31-44-220.ap-southeast-2.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Sep 07 03:25:54 ip-172-31-44-220.ap-southeast-2.compute.internal httpd[11062]: Server configured, listening on: port 80
Sep 07 03:25:54 ip-172-31-44-220.ap-southeast-2.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
ec2-user@ip-172-31-44-220:~$ ls -la /var/www/html/
total 276
drwxr-xr-x. 5 apache apache 16384 Sep  7 03:33 .
drwxr-xr-x. 4 root  root   512 Sep  7 03:25 ..
-rwxr-xr-x. 1 apache apache 523 Sep  6 04:35 .htaccess
-rwxr-xr-x. 1 apache apache 405 Sep  5 16:54 index.php
-rwxr-xr-x. 1 apache apache 19903 Sep  5 16:54 license.txt
-rwxr-xr-x. 1 apache apache 7407 Sep  5 16:54 readme.html
-rw-r--r--. 1 root  root    24 Sep  7 03:33 test.php
-rwxr-xr-x. 1 apache apache 7718 Sep  5 16:54 wp-activate.php
-rwxr-xr-x. 9 apache apache 16384 Sep  5 16:54 wp-admin
-rwxr-xr-x. 1 apache apache 351 Sep  5 16:54 wp-blog-header.php
-rwxr-xr-x. 1 apache apache 2323 Sep  5 16:54 wp-comments-post.php
-rwxr-xr-x. 1 apache apache 3339 Sep  5 16:54 wp-config-sample.php
-rwxr-xr-x. 1 apache apache 3427 Sep  6 03:04 wp-config.php
drwxr-xr-x. 5 apache apache 67 Sep  6 04:35 wp-content
-rwxr-xr-x. 1 apache apache 5617 Sep  5 16:54 wp-cron.php
```

Figure 49: Checking that the application files were restored correctly.

```
ec2-user@ip-172-31-44-220:~$ sudo grep -E "DB_NAME|DB_USER|DB_HOST" /var/www/html/wp-config.php
define( 'DB_NAME', 'wordpress_db' );
define( 'DB_USER', 'wp_user' );
define( 'DB_HOST', 'task-32-db.cteoacm2c5t2.ap-southeast-2.rds.amazonaws.com' );
ec2-user@ip-172-31-44-220:~$ sudo dnf install -y nmap-ncat
Last metadata expiration check: 0:10:21 ago on Mon Sep  7 03:25:39 2026.
Dependencies resolved.
```

Figure 50: Verifying that the database configurations are correct.

```
ec2-user@ip-172-31-44-220:~$ nc -vz task-32-db.cteoacm2c5t2.ap-southeast-2.rds.amazonaws.com 3306
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: TIMEOUT.
ec2-user@ip-172-31-44-220:~$ nc -vz task-32-db.cteoacm2c5t2.ap-southeast-2.rds.amazonaws.com 3306
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Connected to 172.31.48.21:3306.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.
```

Figure 51: An initial attempt to connect to the RDS database timed out. The second attempt showed a successful connection after the problem had been fixed.

The RDS instance's security group was allowing MySQL traffic on port 3306 from another security group that was not configured with the applications instances. This was fixed by adding the correct inbound security rule.

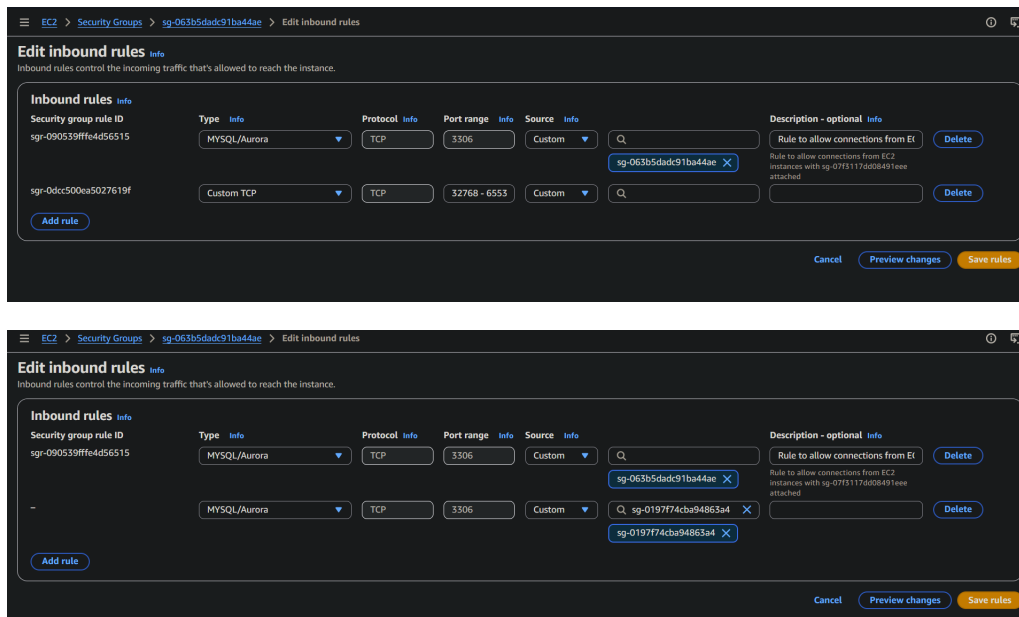


Figure 52: Adding the correct security rule so that the instances could connect to the RDS database.

The health checks now return a "Healthy" status for both instances, confirming that the ALB can communicate with them correctly. The application can be accessed through the ALB's DNS name at <http://task-3-3-alb-2099008439.ap-southeast-2.elb.amazonaws.com/>.

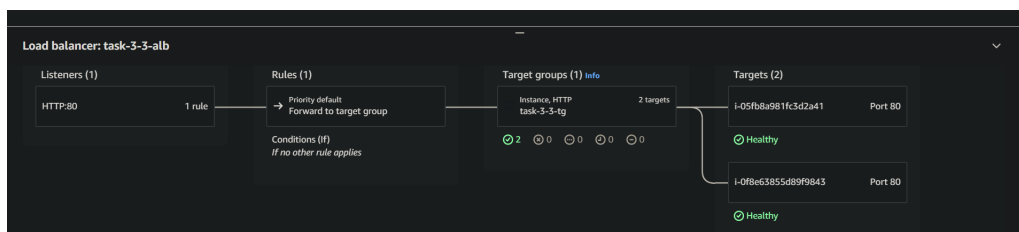


Figure 53: Health checks return "Healthy" status.



Figure 54: The application is running live and can be accessed through the ALB.

2.4 Task 3.4

For task 3.4, I configured a CloudWatch alarm for the Auto Scaling Group to monitor CPU utilisation and trigger when it exceeds 70% over a 5-minute period. An Amazon SNS topic was configured to send an email notification to my student email address when the alarm is triggered. I also enabled AWS Systems Manager (SSM) Session Manager for the EC2 instances, allowing secure terminal access through the AWS Console without requiring public SSH access or open port 22.

2.4.1 CloudWatch Monitoring and Alarm

Before creating the alarm, I set up an SNS topic called `task-3-4-topic`. The CloudWatch alarm uses this topic as the destination to send its notifications, publishing a message when the alarm is triggered by a pre-defined metric. Subscribers to the SNS topic can then receive those notifications.

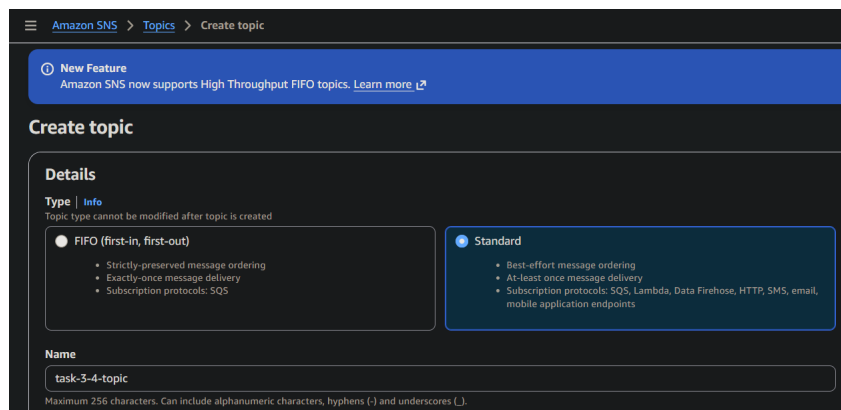


Figure 55: Creating a new SNS topic `task-3-4-topic`.

I then subscribed to this topic so that I could receive an email whenever the CloudWatch alarm gets triggered.

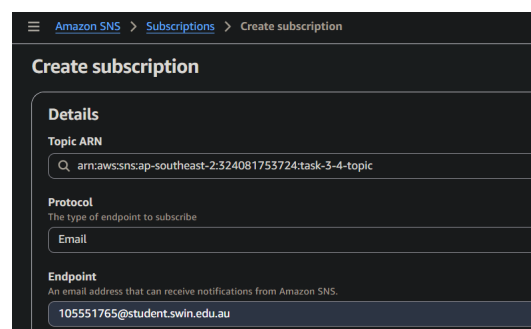


Figure 56: Subscribing to the SNS topic.

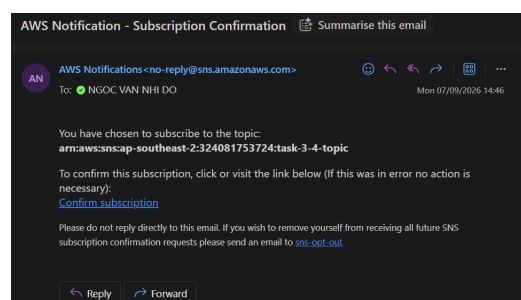


Figure 57: An email is sent to my student email address to verify the subscription.

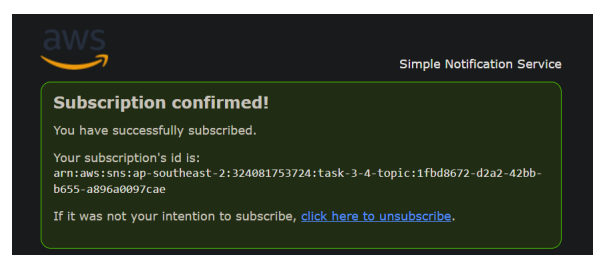


Figure 58: My subscription is verified.

The next step was creating the CloudWatch alarm itself. I set up an alarm on the `CPUUtilization` metric for my ASG `task-3-3` rather on the individual EC2 instances. I also set the threshold for the alarm to "greater than 70", meaning that the alarm is triggered with CPU utilisation exceeds 70% for over 5 minutes.

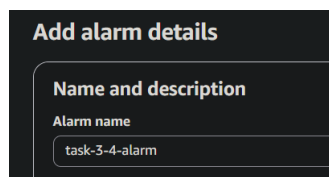


Figure 59: Creating a new CloudWatch alarm `task-3-4-alarm`.

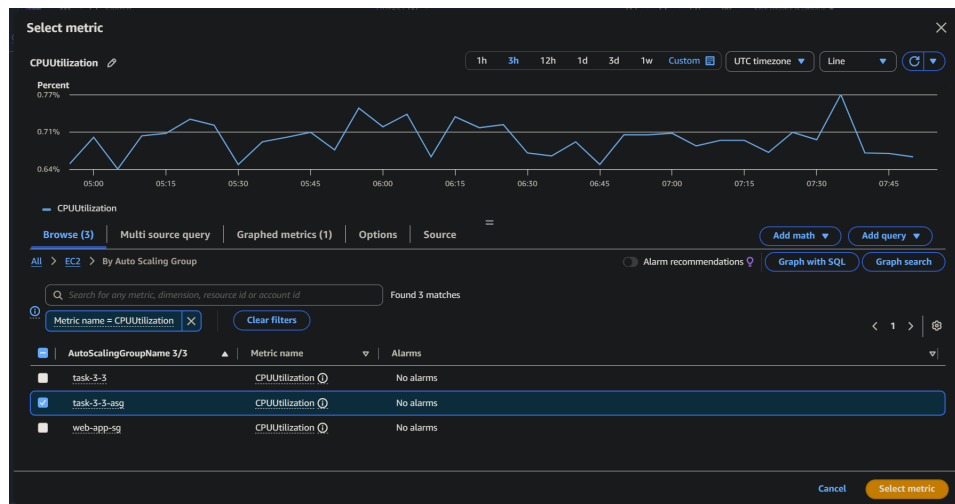


Figure 60: Selecting the `CPUUtilization` metric for my ASG `task-3-3`.

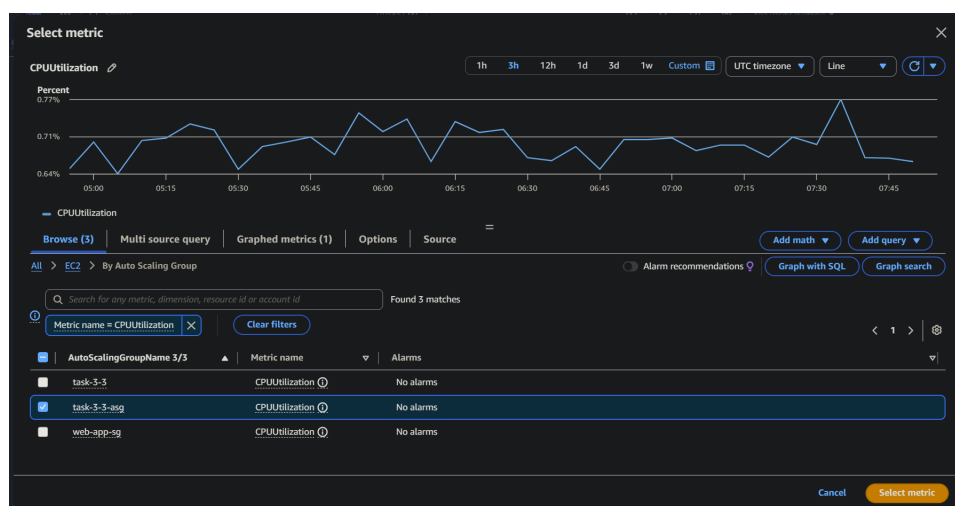


Figure 61: Configuring the threshold for the metric to "greater than 70%".

As previously mentioned, the SNS topic `task-3-4-topic` is attached to the alarm so that I could be notified when the average CPU utilisation metric goes past the defined threshold.

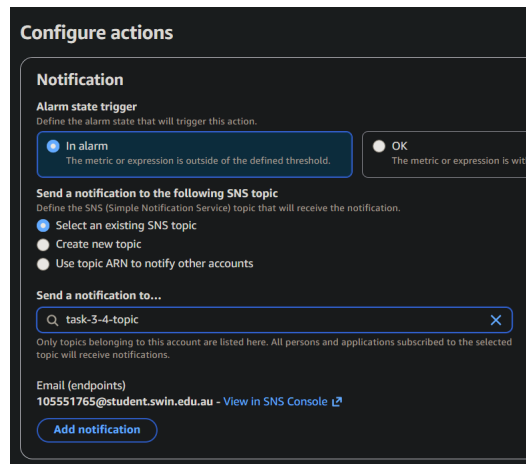


Figure 62: Configuring the metric to Average over a 5-minute period, with the threshold set to Greater than a value of 70.

The alarm has been created. The graph displays the defined threshold as a red dotted line against the current CPU utilisation.



Figure 63: The `task-3-4-alarm` dashboard.

2.4.2 Secure Management via SSM

The goal of this task is to securely access the backend EC2 instances without requiring a private key file or exposing port 22 to the public internet. The first step was to ensure that the EC2 instances had access to the AWS Systems Manager service. This was achieved by attaching the `AmazonSSMManagedInstanceCore` to their IAM role.

I attached this policy to the existing IAM role that was already associated with my EC2 instances.

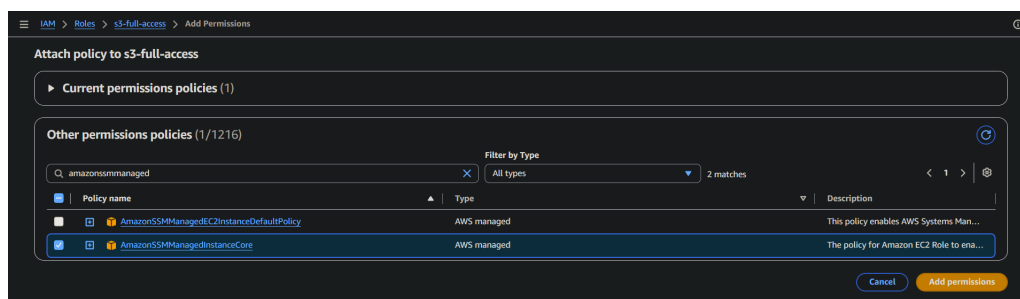


Figure 64: Attaching the policy `AmazonSSMManagedInstanceCore` to the existing IAM role `s3-full-access` (I should've picked a better role name, in retrospect).

As an additional verification step, I SSH-ed into one of the instances to confirm that the SSM agent had already been installed. This is expected, as the SSM agent should be pre-installed on Amazon Linux instances.

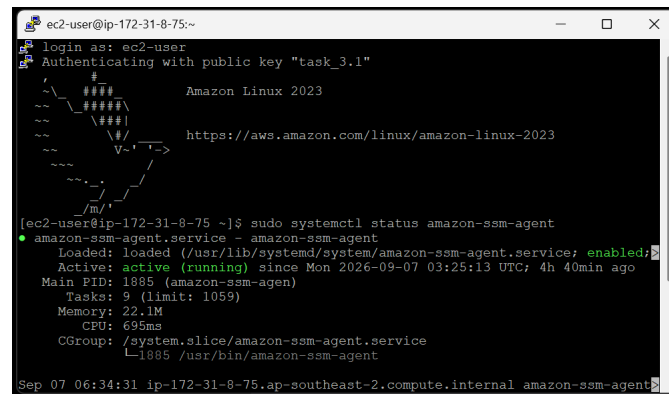


Figure 65: Verifying that the SSM agent already exists on the backend EC2 instances.

Once the role has been modified, the instances are now connected to the Session Manager.

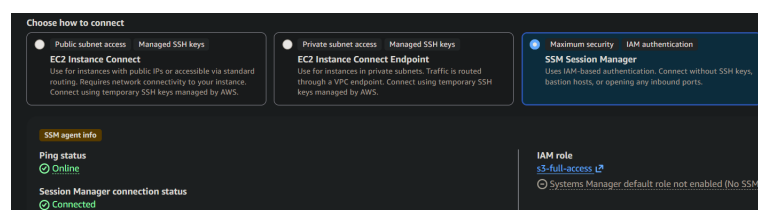


Figure 66: The details of the instance `i-0f8e63855d89f9843` show that it is connected to the Session Manager.

As a proof of concept, I removed the inbound security rule which allowed public SSH access to the EC2 instances. An attempt to SSH into the instance above using PuTTY timed out following this removal.

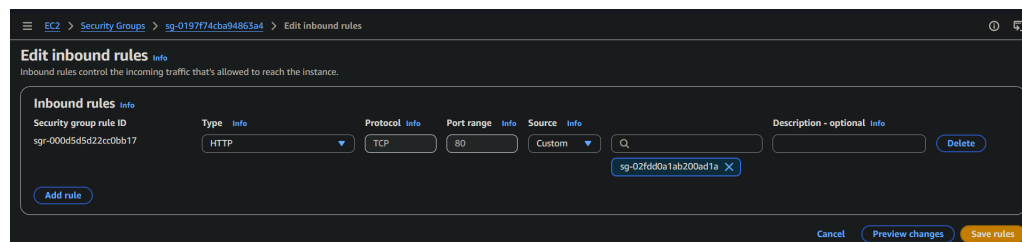


Figure 67: Editing the instances' security group to remove public SSH access.

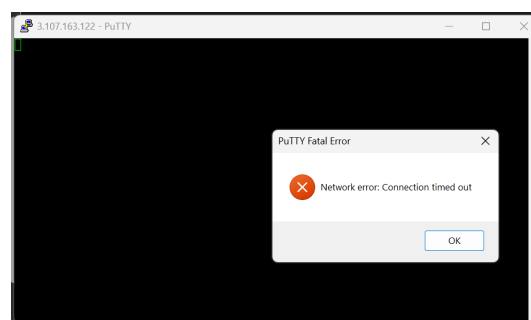


Figure 68: An attempt to SSH into the instance using PuTTY failed.

I then established a connection to the EC2 instance using the Session Manager through the AWS Console to demonstrate that I could securely access the instance's terminal without requiring an SSH connection or the use of a private key.

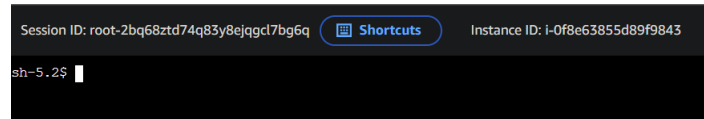


Figure 69: The Session Manager terminal session for the instance `i-0f8e63855d89f9843`.

3 Self-troubleshooting